

Zadania

niedziela, 25 stycznia 2026 09:45

Podatność SQL Injection może pozwalać na: *

- ☒ nieautoryzowanym dostępem w trybie odczytu lub zapisu do całej bazy danych
- ☒ zmniejsić mechanizmu uwierzytelnienia
- ☒ możliwością wykonania kodu w systemie operacyjnym
- ☒ możliwością odczytania wybranych plików

Które algorytmy są dostępne w bibliotece OpenSSL do generowania funkcji skrótu? (2 odpowiedzi)

- ☒ MD5
- ☒ SHA-256
- ☐ ROT13
- ☐ Base64
- ☐ XOR

Olej znaków "aasb763jab" został zapisany do pliku file.txt za pomocą polecenia "echo "aasb763jab" > file.txt". Aby wygenerować skrót sha-256 dla ciągu "aasb763jab" należy wykonać polecenie:

- ☐ echo "aasb763jab" | sha256sum
- ☐ cat file.txt | sha256sum
- ☒ echo -n "aasb763jab" | sha256sum
- ☐ sha256sum file.txt
- ☐ sha256sum < file.txt

Odbiorca (B) w celu weryfikacji podpisanej prze nadawcę (A) wiadomości: *

- ☐ wykorzysta klucz prywatny A
- ☐ wykorzysta klucz prywatny B
- ☒ wykorzysta klucz publiczny A
- ☐ wykorzysta klucz publiczny B

Które polecenia są poprawne w kontekście tworzenia pliku o rozmiarze 1MiB z losową zawartością? (2 odpowiedzi)

- ☒ head -c 1048576 /dev/urandom
- ☒ dd if=/dev/urandom of=file.txt bs=1M count=1
- ☐ cat /dev/random > file.txt
- ☐ echo "1MB" > file.txt
- ☐ truncate -s 1M file.txt

Base64 to: *

- ☐ struktura system plików
- ☒ popularny format kodowania kluczy w kryptografii asymetrycznej
- ☐ protokół warstwy transportowej modelu ISO
- ☒ rodzaj kodowania transportowego
- ☐ model architektury systemów 64 bitowych

Na podstawie poniższego obrazka można powiedzieć, że szczegółowe informacje o procesie xfce4-session można znaleźć w katalogu:

```
PID TTY STAT TIME COMMAND
797 ? Ss 0:00 /lib/systemd/systemd --user
798 ? S 0:00 (sd-pam)
817 ? Sscl 0:00 /usr/bin/xdmcp-receive
818 ? Sscl 0:00 /usr/bin/xdmcp-receive media-session
819 ? Sscl 0:00 /usr/bin/xdmcp-receive --daemonize --log-target=jou
823 ? Ss 0:00 /usr/bin/gnome-keyring-daemon --session --address=systemd
824 ? Ss 0:00 /usr/bin/gnome-keyring-daemon --daemonize --login
825 ? Ss 0:00 xfce4-session
876 ? Ss 0:00 /usr/bin/ssh-agent x-session-manager
```

☐ /dev/proc/828

☒ /proc/828/

☐ /proc/xfce4

☐ /opt/828/x

☐ /var/log/xs

☐ /usr/local

Zaloguj się, by kontynuować

Aby wypieć ten formularz, musisz się zalogować.

[Zgłoś nadużycie](#)

ZALOGUJ SIĘ

Dróg do udanego ataku typu SQLInjection może być wprowadzona w panelu logowania, w polu „Nazwa użytkownika” fraza:

- ☒ alan" -- a
- ☐ alan -- and select password from passwords
- ☐ alan -- a
- ☐ alan union select password for user alan
- ☐ alan and sleep(3)

SQL Injection **może** (w zależności od uprawnień konta DB i konfiguracji) prowadzić do wszystkich rzeczy z listy — więc tutaj zaznaczyłabym **wszystkie 4**. Wyjaśnienie po kolei:

- ☒ **nieautoryzowany dostęp (odczyt/zapis) do bazy danych**
- Najczęstszy skutek: napastnik może odczytywać rekordy (np. użytkowników), modyfikować je, usuwać, czasem wykonywać dodatkowe zapytania.
- ☒ **ominięcie mechanizmu uwierzytelnienia**
- Klasyk: Jeśli logowanie buduje zapytanie SQL z wejścia użytkownika, można doprowadzić do „zalogowania bez hasła” poprzez zmianę warunku w zapytaniu.
- ☒ **możliwość wykonania kodu w systemie operacyjnym**
- To zależy od silnika i konfiguracji, ale bywa możliwe np. przez niebezpieczne funkcje/procedury (albo mechanizmy typu uruchamianie poleceń, extended procedures) i odpowiednie uprawnienia DB. Nie zawsze, ale **może**.
- ☒ **możliwość odczytania wybranych plików**
- Również zależy od bazy i uprawnień, ale część DB ma funkcje/komendy do czytania plików z serwera bazy (np. do importu), co przy złej konfiguracji może zostać nadużyte.

Jeśli to test jednokrotnego wyboru „co na pewno”, to najpewniejsze i najbardziej uniwersalne byłyby pierwsze dwie (DB read/write + auth bypass). Ale przy sformułowaniu „**może pozwalać**” poprawne są wszystkie cztery.

Poprawne 2 odpowiedzi to:

- ☒ MD5
- ☒ SHA-256

Wyjaśnienie:

- MD5 i SHA-256 to kryptograficzne funkcje skrótu (hash functions) i są dostępne w OpenSSL (np. openssl dgst -md5 i openssl dgst -sha256).
- ROT13 to prosta zamiana liter (szyfr podstawieniowy), nie hash.
- Base64 to kodowanie, nie funkcja skrótu (da się odwrócić).
- XOR to operacja bitowa używana w szyfrowaniu/obfuskacji, ale nie jest funkcją skrótu.

Kluczowy szczegół: polecenie echo "aasb763jab" > file.txt zapisuje do pliku tekst + znak nowej linii na końcu (bo domyślnie echo dopisuje \n). Więc w file.txt jest tak naprawdę: aasb763jab\n Pytanie jednak brzmi: skrót dla ciągu aasb763jab (bez dopisanego enterem \n). Dlatego poprawne będzie polecenie, które hashuje dokładnie te znaki bez nowej linii: echo -n "aasb763jab" | sha256sum Wyjaśnienie pozostałych:

- echo "aasb763jab" | sha256sum – tu echo doda \n, więc hash będzie dla aasb763jab\n
- cat file.txt | sha256sum – hashuje zawartość pliku, czyli też aasb763jab\n
- sha256sum file.txt – j.w., hash pliku z \n
- sha256sum < file.txt – j.w., hashuje bytes z pliku (z \n)

Jeśli test jest „podchwytliwy” i chcą hash **dokładnie tego, co jest w pliku po echo**, wtedy poprawne byłyby: sha256sum file.txt albo sha256sum < file.txt (bo one uwzględnią \n). Ale przy sformułowaniu „dla ciągu aasb763jab” — wybór to echo -n ... | sha256sum.

Poprawna odpowiedź to:

- ☒ **wykorzysta klucz publiczny A"**

Wyjaśnienie (krótko i konkretnie)

- Nadawca A podpisuje wiadomość **swoim kluczem prywatnym A**.
- Odbiorca B weryfikuje podpis używając **klucza publicznego A**, bo tylko on pasuje do prywatnego A i pozwala sprawdzić, że:
 1. podpis faktycznie zrobił A (autentyczność),
 2. wiadomość nie została zmieniona (integralność).

Dlaczego pozostałe są błędne

- ~~X~~klucz prywatny A — odbiorca nie powinien go znać
- ~~X~~klucz prywatny B — nie ma związku z podpisem A
- ~~X~~klucz publiczny B — też nie pasuje do podpisu A

Jeśli na Twoim teście obok „klucz publiczny A” jest czerwony X, to możliwe, że pytanie dotyczyło **szyfrowania dla odbiorcy** (tam byłby klucz publiczny B), ale przy sformułowaniu „weryfikacji podpisanej przez A wiadomości” zawsze jest **publiczny A**.

Dwie sensowne komendy do utworzenia pliku 1 MiB z losową zawartością to:

- ☒ **dd if=/dev/urandom of=file.txt bs=1M count=1**
 - if=/dev/urandom daje losowe bajty
 - bs=1M count=1 zapisuje dokładnie 1 MiB do file.txt
- ☒ **head -c 1048576 /dev/urandom > file.txt** (*uwaga: potrzebne jest przekierowanie do pliku*)
 - < 1048576 pobiera dokładnie 1 048 576 bajtów (= 1 MiB)
 - > file.txt zapisuje wynik do pliku

Bez > file.txt ta komenda tylko wypisze dane na ekran i **nie utworzy pliku**.

Pozostałe opcje są niepoprawne w tym kontekście, bo:

- cat /dev/random > file.txt — nie kończy się po 1 MiB i może się blokować (biera „prawdziwą” entropię)
- echo "1MB" > file.txt — zapisze tekst „1MB”, nie 1 MiB danych i nie losowe
- truncate -s 1M file.txt — ustawi rozmiar 1 MiB, ale zawartość nie będzie losowa (zwykle zera / plik rzadki)

Jeśli w Twoim teście pierwsza opcja jest dostawiona bez > file.txt, to formalnie nie spełnia „tworzenia pliku” — ale w praktyce to właśnie ona jest drugim poprawnym sposobem **po dodaniu przekierowania**.

Base64 to kodowanie binarnych danych na znaki tekstowe (A–Z, a–z, 0–9, +, / oraz ewentualnie = jako padding). Nie jest to szyfrowanie ani kompresja — tylko sposób zapisu/przesyłu binarnych bajtów w formie tekstu.

- ☒ „**rodzaj kodowania transportowego**” — TAK
 - Base64 powstało właśnie po to, żeby **dało się przenosić dane binarne** (np. obrazek, klucz, plik) w systemach, które oczekują **tekstu** (np. e-mail, JSON, XML).
 - Przykład: załączniki w e-mailach (MIME), dane w JSON Web Token (często base64url), itp.
- ☒ „**popularny format kodowania kluczy w kryptografii asymetrycznej**” — TAK
 - Klucze i certyfikaty często są w formatach, które zawierają Base64.
 - Najprostsz przykład: PEM (—BEGIN CERTIFICATE—...) — śródek to **Base64** danych DER.
 - To nie znaczy, że Base64 *jest* kryptografią — tylko, że **klucze/certyfikaty są często tak zapisywane** w plikach tekstowych.
- ~~X~~ „**struktura systemu plików**” — NIE
 - To by dotyczyło rzeczy typu ext4, NTFS, FAT itp.
 - Base64 nie ma nic wspólnego z organizacją danych na dysku.
- ~~X~~ „**protokół warstwy transportowej modelu ISO**” — NIE
 - Warstwa transportowa to np. TCP/UDP (w praktyce modele TCP/IP).
 - Base64 nie jest protokołem sieciowym — to tylko kodowanie danych.
- ~~X~~ „**model architektury systemów 64 bitowych**” — NIE
 - „64” w Base64 odnosi się do **liczby znaków w alfabecie (64 symbole)**, a nie do 64-bitowego procesora.

Poprawna odpowiedź to:

- ☒ **/proc/828/**

Dlaczego: na obrazku widać listę procesów i że **xfce4-session ma PID = 828**. W Linuksie szczegółowe informacje o procesie (m.in. cmdline, environ, cwd, exe, fd/, status, maps) są w pseudo-systemie plików **/proc/<PID>/**, czyli tutaj **/proc/828/**.

Pozostałe propozycje są błędne, bo:

- /dev/proc/828 – taka standardowa ścieżka nie istnieje
- /proc/xfce4-session – /proc jest indeksowane PID-ami, nie nazwami procesów
- /opt/...i /var/log/... – to nie jest miejsce na metadane procesu (logi mogą być w /var/log, ale nie “szczegółowe informacje o procesie” w sensie /proc).

Żeby wstrzyknąć ię z pola tekstowego zadziałało w typowym (podatnym) logowaniu, zwykle muszą zajść naraz rzeczy typu:

1. **Wyciąć z literału tekstowego** (dominięcie cudzoziemcy, którym aplikacja otacza input – najczęściej „, rzadziej ”).
2. Dodanie jakiegoś **operatora/układni SQL** poza stringiem.
3. Często też **zneutralizowanie reszty zapytania** komentarzem (– ..., / * ... */), bo aplikacja dopisuje np. dalsze warunki.

Jeśli nie ma kroku (1), to reszta (–, union, select, sleep) jest zwykle **tylko częścią tekstu username** i nie „wykonuje się”.

1) alan" -- a

Co to próbuje zrobić: zakończyć string przez " i „uciąć resztę” komentarzem -- a.

Czy ma szansę zadziałać?

Tylko wtedy, gdy aplikacja **naprawdę używa " jako ogranicznika stringów** w tym miejscu (np. skleja ... username="WEIŚCIE" ...).

Która z grup kodów odpowiedzi HTTP oznacza, że coś poszło nie tak po stronie serwera?

☐ 200

☐ 300

☐ 400

☒ 500

można powiedzieć, że:

~/.ssh/known_hosts:1:~/.Downloads:
[~] openssl x509 -in www-debian-org.1.pem -text
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
012b0884a4cf8c18a7531f8062e25a75f5a
Signature Algorithm: sha256withRSAEncryption
Issuer: C = US, O = Internet Security Research Group, CN = ISRG Root X1
Validity
Not Before: Sep 4 00:00:00 2020 GMT
Not After : Sep 15 16:00:00 2025 GMT
Subject: C = US, O = Let's Encrypt, CN = R3
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:bb:82:15:28:cc:f8:a0:94:d3:0f:12:0c:8d:55:
92:c3:f8:02:f1:99:a0:7a:e4:00:a7:3d:2a:ae:1b:
20:9f:c5:4c:3b1c4f4e:de:f6:75:c8:b3:c7:0f:07:47:

[~] wystawca certyfikatu jest "Let's Encrypt"
[~] wystawca certyfikatu jest "Internet Security Research Group"
[~] wystawca certyfikatu jest "debian.org"
[~] klucz publiczny "Let's Encrypt" ma rozmiar 2048 bitów
[~] Klucz publiczny "Internet Security Research Group" ma rozmiar 2048 bitów

~/.ssh/known_hosts:1:~/.Downloads:
[~] openssl x509 -in www-debian-org.1.pem -text
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
012b0884a4cf8c18a7531f8062e25a75f5a
Signature Algorithm: sha256withRSAEncryption
Issuer: C = US, O = Internet Security Research Group, CN = ISRG Root X1
Validity
Not Before: Sep 4 00:00:00 2020 GMT
Not After : Sep 15 16:00:00 2025 GMT
Subject: C = US, O = Let's Encrypt, CN = R3
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:bb:82:15:28:cc:f8:a0:94:d3:0f:12:0c:8d:55:
92:c3:f8:02:f1:99:a0:7a:e4:00:a7:3d:2a:ae:1b:
20:9f:c5:4c:3b1c4f4e:de:f6:75:c8:b3:c7:0f:07:47:

Z tego, co widać na zrzucie z openssl x509 -in-text, da się odczytać dwie kluczowe linie:

- **Issuer:** O = Internet Security Research Group, CN = ISRG Root X1
- **Subject:** O = Let's Encrypt, CN = R3
- oraz **Public-Key:** (2048 bit) w sekcji **Subject Public Key Info** (czyli klucz podmiotu/Subject, nie wystawcy).

Co z tego wynika (poprawne odpowiedzi)

☒ **wystawcą certyfikatu jest "Internet Security Research Group"**
Bo Issuer to wystawca.

☒ **Klucz publiczny "Let's Encrypt" ma rozmiar 2048 bitów"**
Bo Subject to Let's Encrypt (R3), a rozmiar klucza publicznego pokazany w certyfikacie dotyczy właśnie Subject (widać (2048 bit)).

Co jest błędne / nie wynika z obrazka

☒ **wystawcą certyfikatu jest "Let's Encrypt"** – Let's Encrypt jest tu Subject, a nie Issuer.

☒ **Klucz publiczny "Let's Encrypt" – nigdzie tego nie ma w Issuer/Subject.**

☒ **Klucz publiczny "Internet Security Research Group" ma rozmiar 2048 bitów"** – na tym wydruku widzisz klucz Subject, nie klucz wystawcy. Z samego tego certyfikatu nie wyciągniesz rozmiaru klucza issuer (to byłby w certyfikacie wystawcy/roota).

Jeśli chcesz, mogę Ci też powiedzieć proste „hasło” na test: **Issuer = wystawca, Subject = dla kogo jest certyfikat**.

W systemach Unix/Linux w celu zwiększenia bezpieczeństwa:*

☐ hasła użytkowników są wstępnie szyfrowane i zapisywane na dysku

☒ na dysku zapisywane są **tylko** funkcje skrótu hasel

☐ hasła są zapisywane w jawnej postaci w pliku /etc/shadow

☐ hasła są zapisywane w katalogu domowym użytkownika root

☐ hasła użytkowników nie są nigdzie zapisywane

Prawna odpowiedź to:

☒ **na dysku zapisywane są jedynie funkcje skrótu hasel"**

Wyjaśnienie

W Unix/Linux hasła **nie są przechowywane w jawnej postaci**. System zapisuje w /etc/shadow (zwykle) **hash** hasła wraz z parametrami (np. solą i informacją o algorytmie). Przy logowaniu system hashuje podane hasło i porównuje z tym, co jest zapisane.

Czemu pozostałe są błędne

☒ **„hasła są wstępnie szyfrowane i zapisywane”** — to nie szyfrowanie (odwracalne), tylko **hashowanie** (jednakierunkowe).

☒ **„hasła są w jawnej postaci w /etc/shadow”** — nie, są tam hashe.

☒ **„w katalogu domowym roota”** — nie.

☒ **„hasła nie są nigdzie zapisywane”** — są (w postaci hashy) właśnie po to, by dało się je weryfikować.

Które tryby szyfrowania są dostępne dla algorytmu AES? (3 odpowiedzi) *

☒ ECB

☒ CBC

☒ CFB

☐ ROT

☐ BASE

Znaczyć te 3 tryby AES:

☒ ECB

☒ CBC

☒ CFB

Wyjaśnienie:

☒ **ECB (Electronic Codebook), CBC (Cipher Block Chaining) i CFB (Cipher Feedback)** to klasyczne **tryby pracy szyfru blokowego** (AES jest szyfrem blokowym).

☒ **ROT** to nie tryb AES, tylko proste szyfry przesuwne (np. ROT13).

☒ **BASE** nie jest trybem szyfrowania (kojarzy się raczej z Base64, czyli kodowaniem).

Poniżej przedstawiono fragment podczepnika systemowego dla polecenia openssl. *

Szyfrowanie pliku data.txt algorytmem aes w trybie wiązania zasyfrowanych bloków, z blokami o rozmiarze 256 bitów z hasłem "as34Dw9P2iSh" przedstawia polecenie:

openssl enc -ciphertext [-AddPp] [-base64] [-bufsize number] [-debug]
[-in file] [-iv IV] [-k key] [-k passord] [-kfile file]
[-md digest] [-nomd] [-nopad] [-nosalt] [-out file] [-pass arg]
[-S salt] [-salt]

☒ openssl enc -e -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2iSh

☐ openssl enc -e -aes-256-ecb -in data.txt -out data.enc -k as34Dw9P2iSh

☐ openssl enc -d -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2iSh

☐ openssl enc -d -aes-256-ecb -in data.txt -out data.enc -k as34Dw9P2iSh

☐ openssl enc

Zalozmy dla kolumny hasla

Z opisu: **„tryb wiązania zasyfrowanych bloków” = CBC (Cipher Block Chaining)**, a **„bloki 256 bitów”** w tym kontekście oznacza **AES-256** (długość klucza 256 bit, bo AES ma zawsze blok 128-bitowy).

Czyli poprawna komenda (szyfrowanie, nie deszyfrowanie) to ta z:

☒ enc -e (encrypt)

☒ -aes-256-cbc

☒ -in data.txt -out data.enc

☒ hasło podane jako -k as34Dw9P2iSh (hasło), a nie -k (hex-klucz)

openssl enc -e -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2iSh

Dlaczego pozostałe widoczne odpadają

☒ ... -aes-256-ecb ... ☒ Bo ECB to nie CBC

☒ ... -d -aes-256-cbc ... ☒ Bo -d to deszyfrowanie

☒ ... -K as34... ☒ Bo -K oczekuje klucza w hex, a tu jest hasło (tekst)

Jeśli brakująca odpowiedź była zasłonięta, to i tak szukaj tej, która dokładnie odpowiada wzorcowi powyżej: **-e -aes-256-cbc ... -k <hasło>**.

W kryptografii asymetrycznej: *

☐ wysyłane wiadomości szyfruje się kluczami publicznymi nadawcy

☐ wysyłane wiadomości szyfruje się kluczami prywatnymi nadawcy

☒ wysyłane wiadomości szyfruje się kluczami publicznymi odbiorcy

☐ wysyłane wiadomości szyfruje się kluczami prywatnymi odbiorcy

Prawna odpowiedź (dla szyfrowania „żeby tylko odbiorca mógł przeczytać”) to:

☒ **wysyłane wiadomości szyfruje się kluczami publicznymi odbiorcy**

Wyjaśnienie

W kryptografii asymetrycznej do poufności działa to tak:

☒ Nadawca bierze **klucz publiczny odbiorcy** i nim szyfruje wiadomość.

☒ Odbiorca odszyfrowuje ją swoim **kluczem prywatnym**.

Pozostałe opcje są błędne w kontekście poufności:

☒ ☒ **Publiczny nadawcy** – każdy go zna, więc nie zapewnia poufności dla odbiorcy

☒ **Prywatny nadawcy** – to byłby raczej mechanizm podpisu (w uproszczeniu), nie standardowe szyfrowanie do poufności

☒ **Prywatny odbiorcy** – prywatnym kluczem się nie „wysyła” do szyfrowania; on jest do odszyfrowania i powinien pozostać tajny

Które elementy kodu mogą prowadzić do podatności XSS? *

echo "<div>Welcome". \$_GET[name]. "</div>";
echo "<script>var data = ". \$_POST[data]. "</script>";
document.write(location.hash.substring(1));
\$('#div').html(userinput);

☒ Bezpośrednie użycie \$_GET[name] w HTML

☒ Wstawianie \$_POST[data] do JavaScript

☐ Użycie protokołu HTTP

☐ Brak walidacji długości danych

☒ Używanie jQuery.html(userinput)

Zaloguj się, by kontynuować

W tym pytaniu „elementy kodu” prowadzące do XSS to dokładnie te miejsca, gdzie **dane od użytkownika** trafiają do **HTML/JS bez escape/sanitizacji**. Z widocznych checkboów zaznacz:

☒ **Bezpośrednie użycie \$_GET[name] w HTML**

☒ **Wstawianie \$_POST[data] do JavaScript**

☒ **ta ostatnia ucięta** Używanie jQuery.html(userinput) (to praktycznie jak innerHTML, bardzo częsty wektor XSS)

I nie zaznaczać:

☒ **Użycie protokołu HTTP** (to nie jest przyczyna XSS)

☒ **Brak walidacji długości danych** (długość nie rozwiązuje XSS; potrzebny jest escape/sanitizacja)

Dodatkowo: w samym fragmencie kodu jest jeszcze document.write(location.hash.substring(1)); — jeśli wśród odpowiedzi masz opcję odnoszącą się do **document.write/location.hash**, to też ją zaznacz (to również typowy DOM XSS).

Bo XSS powstaje wtedy, gdy **dane kontrolowane przez użytkownika** trafiają do **kodu HTML** albo **JavaScript** w taki sposób, że przeglądarka zaczyna je interpretować jako znacznik/JS, a nie jako zwykły tekst. W Twoich przykładach dokładnie to się dzieje.

1) \$_GET[name] w HTML

☒ echo "<div>Welcome". \$_GET[name]. "</div>";

☒ \$_GET[name] pochodzi z URL (użytkownik może to zmienić).

☒ Wklejasz to **wprost do HTML** bez escapingu (np. bez htmlspecialchars()).

☒ Jeśli ktoś wstawi tam np. fragment HTML, przeglądarka może go potraktować jako prawdziwy kod HTML/JS → XSS.

2) \$_POST[data] w JavaScript

☒ echo "<script>var data = '". \$_POST[data]. "'</script>";

☒ \$_POST[data] pochodzi z formularza/ządania (też kontrolowane przez użytkownika).

☒ Wkładasz to do **kontekstu JavaScript** w cudzysłowie '...'.

☒ Jeśli ktoś wstrzyknie znaki, które „wychodzą” z tego cudzysłowu i dopisują kod, przeglądarka wykona ten kod → XSS.

☒ To jest szczególnie zdradliwe, bo tu potrzebne jest **kodowanie pod kontekst JS**, nie tylko HTML.

3) jQuery.html(userinput)

☒ \$('#div').html(userinput);

☒ .html() ustawia zawartość elementu jako **HTML** (jak innerHTML).

☒ Jeśli userinput zawiera znaczniki, to one zostaną wyrenderowane jako HTML, a nie pokazane jako tekst.

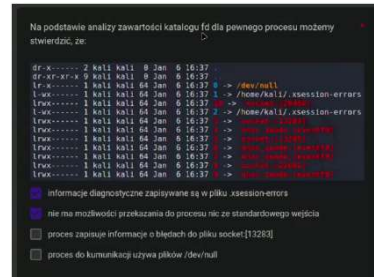
☒ Bez sanitizacji (text()) zamiast html(), albo oczyszczanie HTMLu to klasyczny DOM XSS.

Dlaczego NIE „HTTP” i NIE „walidacja długości”

☒ **„Użycie protokołu HTTP”**: XSS działa zarówno na HTTP, jak i HTTPS. To nie protokół jest problemem, tylko **renderowanie niesprawdzonych danych** w HTML/JS.

☒ **„Brak walidacji długości danych”**: ograniczenie długości nie usuwa XSS — krótki payload też potrafi być groźny. Kluczowe jest **escape/sanitizacja zgodna z kontekstem** (HTML, atribut JS, URL itd.).

Jeśli pokażesz (lub przepiszesz) tę „uciętą” ostatnią odpowiedź, mogę powiedzieć dokładnie, jak ją uzasadnić względem tej konkretnej linii kodu.



Z tego listingu `/proc/<pid>/fd` widać:

- `0` → `/dev/null` (stdin)
- `1` → `/home/kali/.xsession-errors` (stdout)
- `2` → `/home/kali/.xsession-errors` (stderr)
- dalej są różne socket[...] i anon_inode[eventfd]

Dlatego poprawne są **dwie** pierwsze odpowiedzi:
☒ **informacje diagnostyczne zapisywane są w pliku .xsession-errors**
— bo zarówno `stdout` (1) jak i `stderr` (2) idą do `~/xsession-errors`
☒ **nie ma możliwości przekazania do procesu nic ze standardowego wejścia**
— bo `stdin` (0) jest podpięty do `/dev/null`, więc proces nie dostanie realnego wejścia z klawiatury/potoku (odczyt to praktycznie od razu EOF).
A te dwie są **niepoprawne**:

- ✗ proces zapisuje informacje o błędach do pliku socket:[13283]
- ✗ socket:[13283] to po prostu **gniazdo** na fd 3 (komunikacja), ale błędy lecą na fd 2 do `.xsession-errors`.
- ✗ proces do komunikacji używa plików `/dev/null`
- `/dev/null` jest użyte jako **stdin**, nie jako kanał komunikacji.

TLS_ECDHE_ECDSA_WITH_AES256_GCM_SHA384 rozkłada się tak:

- **ECDSA** – uzgadnianie klucza (wymiana) metodą *ephemeral Elliptic-Curve Diffie-Hellman*
- **ECDSA** – uwierzytelnianie/podpis (certyfikat serwera i podpisy w handshake)
- **AES256_GCM** – szyfrowanie danych + integralność w trybie AEAD (GCM daje „MAC/tag” wbudowany)
- **SHA384** – hash używany w TLS 1.2 głównie w **PRF/handshake**, a nie jako osobny MAC dla danych aplikacyjnych (bo to robi GCM)

Zaznacz:

- ☒ **do szyfrowania ... wykorzystywany jest algorytm Diffie-Hellman**
- sensownie chodzi o **wymianę/uzgadnianie klucza** (ECDHE to odmiana Diffie-Hellmana na krzywych eliptycznych).

Nie zaznaczaj:

- ✗ do podpisywania wiadomości wykorzystuje się AES256* (AES to szyfr symetryczny, nie podpis)
- ✗ do wygenerowania MAC wykorzystywana jest SHA384* (przy **AES-GCM** integralność zapewnia GCM, SHA384 nie jest MAC-em danych)
- ✗ do szyfrowania wiadomości wykorzystuje się DSA* (jest ECDSA, nie DSA do szyfrowania)
- ✗ wymienione klucze należą do RSA* (tu jest EC: ECDHE/ECDSA)

PBKDF2 to funkcja **wyprowadzania klucza z hasła** (Key Derivation Function), która robi to celowo **wolniej** przez wiele iteracji, żeby utrudnić zgadywanie hasel.

Dlatego poprawna odpowiedź to:

- ☒ **zwiększeniu kosztów obliczeniowych w celu redukcji podatności na atak typu "brut force"**

Pozostałe są błędne:

- ✗ **zmniejszenie rozmiaru pliku po zaszyfrowaniu** — szyfrowanie zwykle nie kompresuje, często nawet dodaje narzut
- ✗ **wykorzystanie silniejszego algorytmu szyfrującego** — PBKDF2 nie zmienia szyfru (np. AES), tylko sposób uzyskania klucza z hasła
- ✗ **zabezpieczenie przed atakiem „łamaniem algorytmu szyfrującego”** — KDF nie „naprawia” słabości szyfru
- ✗ **redukcja kosztów obliczeniowych wyodrębnienia klucza z hasła** — jest odwrotnie: **podnosi** koszty celowo

Poprawna odpowiedź to:

- ☒ **/var/log**

W większości dystrybucji Linuksa standardowe logi systemowe i usług (np. syslog, messages, auth.log, kern.log, journal-related pliki, logi Apache/Nginx itd.) trzymają się właśnie w **/var/log**.

Pozostałe opcje typu `/tmp/log`, `/logs/`, `/usr/cache/logs`, `/opt/usr/local/logs` nie są standardowymi lokalizacjami logów systemowych.

Polecenie:

```
echo 'password' | shasum -a 256
zrobi SHA-256 z tekstu password + znaku nowej linii (\n), bo zwykłe echo dopisuje newlinę na końcu.
Dlatego wygenerowany skrót (hex, 64 znaki) to:
398f71d9dfecdadbafaf5f638d87e40a6e604e1f8697c4e2baf8bcf772daf8fb
(za 398f71d9dfecdadbafaf5f638d87e40a6e604e1f8697c4e2baf8bcf772daf8fb)
Dla porównania: gdyby było echo -n 'password' | shasum -a 256, hash byłby inny (bez \n).
```

Bo **SHA-256** ma **wynik o długości 256 bitów**, a na screenie wynik jest wypisany w **hexie**.

- 1 znak hex = **4 bity** (bo hex to zapis w systemie 16: 0-f).
- 256 bitów ÷ 4 bity/znak = **64 znaki**.

Można to też policzyć bajtami:

- 256 bitów = **32 bajty**,
- każdy bajt to 2 znaki hex,
- 32 × 2 = **64 znaki**.

W tym pytaniu poprawna jest ta **odpowiedź o "hybrid encryption"** (czyli: losowy klucz symetryczny + szyfrowanie pliku symetrycznie + zaszyfrowanie klucza symetrycznego kluczem publicznym).

☒ **wygenerować losowy klucz... zaszyfrować plik algorytmem symetrycznym... zaszyfrować klucz symetryczny kluczem publicznym...**

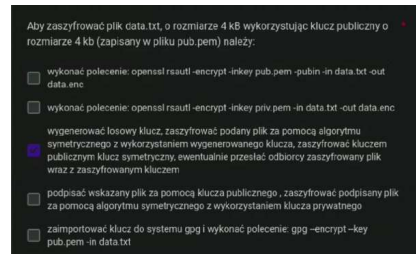
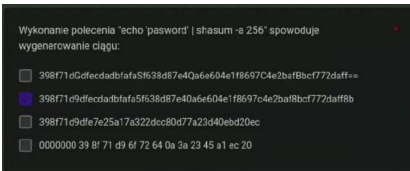
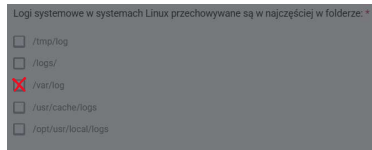
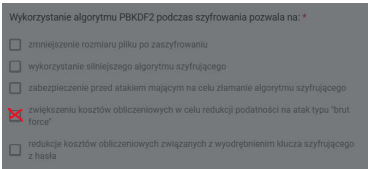
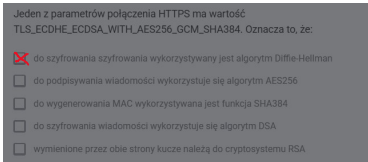
Dlaczego tak?

RSA (nawet z kluczem 4096 bit) **nie służy do szyfrowania dużych plików bezpośrednio**:

- RSA ma ograniczenie rozmiaru danych wejściowych (można zaszyfrować tylko wiadomość krótszą od modułu, i jeszcze mniej przez padding).
- Do plików używa się więc standardowo podejścia **hybrydowego**: szybki szyfr symetryczny do danych + RSA tylko do klucza.

Dlaczego pozostałe odpadają

- ✗ `openssl rsautl -encrypt ... -pubin -in data.txt` — nawet jeśli polecenie jest "logicznie" o publicznym kluczu, **nie zaszyfruje 4 kB** bezpośrednio RSA (za duże).
 - ✗ `rsautl -r priv.pem` — szyfrowanie „prywatnym” to nie jest standardowa poufność (to myli się z podpisem), a poza tym rozmiar dalej problem.
 - ✗ **podpisać** kluczem publicznym... — podpis robi się **prywatnym**, weryfikuje **publicznym**, więc opis jest błędny.
 - `gpg -encrypt ...` — GPG też używa hybrydowego podejścia, ale ta opcja na screenie wygląda na źle sformułowaną (w praktyce podaje się odbiorcę -r, a nie sam plik PEM jako -key pub.pem). W testach zwykle chcą odpowiedź koncepcyjną o hybrydowym szyfrowaniu.
- Jeśli to pytanie wymaga jednej odpowiedzi — wybierz **hybrid encryption** (ta zaznaczona u Ciebie).



W przedstawionej na poniższym rysunku sytuacji mamy do czynienia z: 

```
(kali@kali)-[/bin]
└─$ php8.1 /root/.bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
  *i*) ;;
  *) return;;
esac
```

- ☒ eskalację uprawnień użytkownika
- ☐ wykorzystaniem polecenia "sudo"
- ☒ błędami w nadawaniu uprawnień programom
- ☐ nieudaną próbą dostępu do plików w katalogu domowym użytkownika root
- ☐ zmianą praw dostępu do pliku .bashrc

Na tym screenie widać, że jako użytkownik **kali** uruchamiasz: `php8.1 /root/.bashrc` i **udaje się odczytać** zawartość pliku z katalogu `/root` (widać linie z `.bashrc`). Normalnie użytkownik nie-root **nie ma dostępu** do `/root`, więc to oznacza, że proces `php8.1` działa z podwyższonymi uprawnieniami (np. ma ustawiony SUID/SGID albo jest źle skonfigurowany w systemie). Dlatego poprawnie pasują:
☒ eskalację uprawnień użytkownika
☒ błędami w nadawaniu uprawnień programom
A te nie pasują:
☒ wykorzystaniem polecenia `sudo` (tu go nie ma)
☒ nieudaną próbą dostępu do plików w katalogu domowym `roota` (bo dostęp się udał)
☒ zmianą praw dostępu do pliku `.bashrc` (na screenie nie ma `chmod/chown`, tylko odczyt)
Jeśli to test jednokrotnego wyboru, najbliższa „jedna” odpowiedź to zwykle: **błędy w nadawaniu uprawnień programom** (bo to przyczyna), ale merytorycznie sytuacja pokazuje też eskalację uprawnień (skutek).